

LAB — Utenti, Permessi e File

Modulo: 2B | **Corso:** Lab Amministrazione di Sistemi T

Prerequisiti: Modulo 2A completato (useradd, chmod, umask, SUID/SGID)

VM: `cd ~/sysAdmin-lab && vagrant up --provider=virtualbox && vagrant ssh`

Fonte: LAB “Esempi di gestione utenti, permessi e file [31 marzo]”

Prima di iniziare — cosa trovi in questo lab

Il lab ha 6 esercizi. Il Modulo 2B copre i primi due:

1. **Osservare** `/etc/skel`, `/etc/default/useradd`, `/etc/login.defs`
2. **Creare** una directory collaborativa con SGID per due utenti

Gli esercizi 3–6 (dd, fuser/lsof, script con find) sono nel Modulo 2C.

Esercizio 1 — Osservare i file di configurazione utenti

`/etc/skel`

Quando `useradd -m` crea la home di un utente, copia al suo interno tutto il contenuto di `/etc/skel`. È il “template” della home.

```
ls -la /etc/skel
```

Output tipico su Debian:

```
drwxr-xr-x  2 root root 4096 ... .
drwxr-xr-x 77 root root 4096 ... ..
-rw-r--r--  1 root root  220 ... .bash_logout
-rw-r--r--  1 root root 3526 ... .bashrc
-rw-r--r--  1 root root  807 ... .profile
```

Ogni utente creato con `-m` parte con questi tre file nella propria home. Se vuoi che tutti i nuovi utenti abbiano ad esempio un `.vimrc` preconfigurato, basta aggiungerlo qui:

```
sudo cp ~/.vimrc /etc/skel/.vimrc
# → tutti i nuovi utenti avranno .vimrc già configurato
```

`/etc/default/useradd`

Contiene i valori di default usati da `useradd` quando non specifichi un’opzione:

```
cat /etc/default/useradd
```

Campi principali:

Campo	Esempio	Significato
GROUP	100	GID del gruppo di default (se non si usa -U)

Campo	Esempio	Significato
HOME	/home	Directory genitore delle home
INACTIVE	-1	Giorni dopo scadenza password prima di bloccare l'account (-1 = mai)
EXPIRE	' Data di scadenza account (vuoto = nessuna) SHELL /bin/sh Shell di default (ecco perché si passa sempre-s /bin/bashesplicitamente) SKEL /etc/skel'	Directory template da cui copiare i file

/etc/login.defs

Configurazione più bassa — policy globali di sistema per la gestione degli account:

```
grep -v '^#' /etc/login.defs | grep -v '^$'
```

Campi rilevanti:

Campo	Valore tipico	Significato
PASS_MAX_DAYS	99999	Giorni massimi prima che la password scada
PASS_MIN_DAYS	0	Giorni minimi prima di poter cambiare password
PASS_WARN_AGE	7	Giorni di preavviso prima della scadenza
UID_MIN	1000	UID minimo per utenti fisici (ecco la convenzione 1000+)
UID_MAX	60000	UID massimo per utenti fisici
UMASK	022	Umask di default per le sessioni di login

Connessione con la teoria: questi valori sono i default da cui parte `useradd`. Si sovrascrivono per singolo utente con `chage` (politiche password) o con i parametri di `useradd`/`usermod` alla creazione.

Esercizio 2 — Directory collaborativa con SGID

Obiettivo: creare due utenti (`maria` e `piero`) che possano collaborare in una directory condivisa — creare file leggibili e scrivibili da entrambi — senza toccare i file personali dell'uno o dell'altro.

Passo 1 — Creare gli utenti

```
sudo useradd -m -s /bin/bash maria
sudo useradd -m -s /bin/bash piero
```

```
sudo passwd maria
sudo passwd piero
```

Verifica: dopo questi comandi, ogni utente ha il proprio gruppo privato omonimo. `maria` ha gruppo primario `maria`, `piero` ha gruppo primario `piero`. Se ora creassero file con permessi di gruppo aperti (664), solo l'altro membro del loro rispettivo gruppo potrebbe accedervi — ma `maria` non è nel gruppo `piero` e viceversa. Da soli non possono ancora collaborare.

```
# Conferma: ognuno è l'unico membro del proprio gruppo
grep maria /etc/group
grep piero /etc/group
```

Passo 2 — Allargare la umask

Di default la umask è 0022, che produce file con permessi 644 (il gruppo non può scrivere). Per la collaborazione serve che i file creati siano scrivibili dal gruppo, quindi umask 0002:

```
# Aggiungere umask 0002 al .bashrc di entrambi gli utenti
echo "umask 0002" | sudo tee -a /home/maria/.bashrc
echo "umask 0002" | sudo tee -a /home/piero/.bashrc
```

Con umask 0002, i file creati avranno permessi 664 (rw-rw-r-) e le directory 775 (rwxrwxr-x) — il gruppo può leggere e scrivere.

Passo 3 — Creare il gruppo e aggiungervi gli utenti

```
sudo groupadd programmatori
sudo usermod -aG programmatori maria
sudo usermod -aG programmatori piero
```

Verifica:

```
groups maria    # maria : maria programmatori
groups piero    # piero : piero programmatori
```

Nota: la modifica ai gruppi è attiva alla prossima sessione di login. Se fai `su - maria` subito dopo, il gruppo `programmatori` potrebbe non essere ancora attivo nella sessione. Per forzarlo senza ri-login: `newgrp programmatori`.

Passo 4 — Creare la directory collaborativa con SGID

```
sudo mkdir /home/programmatori
sudo chgrp programmatori /home/programmatori
```

Ora i permessi: vogliamo che owner e gruppo possano fare tutto, gli altri nulla. E soprattutto, vogliamo il **SGID** — così i file creati dentro ereditano automaticamente il gruppo `programmatori` invece del gruppo primario del creatore.

```
sudo chmod 02770 /home/programmatori
```

Il 2 davanti attiva il SGID. Verificare:

```
ls -la /home/
# drwxrws--- 2 root programmatori 4096 ... programmatori
```

```
# ~
# 's' = SGID attivo
```

Perché SGID è fondamentale qui: senza SGID, maria crea un file e il gruppo proprietario è **maria** (il suo gruppo primario). Piero è nel gruppo **programmatori**, non nel gruppo **maria** — non può scrivere quel file. Con SGID, il file viene creato col gruppo **programmatori** — entrambi ci possono accedere.

Passo 5 — Test

```
# Come maria
su - maria
cd /home/programmatori
touch file_di_maria.txt
echo "contenuto di maria" > file_di_maria.txt
ls -la
# -rw-rw-r-- 1 maria programmatori ... file_di_maria.txt
#
# gruppo programmatori, non maria - grazie a SGID
exit

# Come piero
su - piero
cd /home/programmatori
cat file_di_maria.txt      # → "contenuto di maria"
echo "aggiunta di piero" >> file_di_maria.txt  # → funziona
exit

# Come vagrant (fuori dal gruppo)
cd /home/programmatori     # → Permission denied
```

Riepilogo dei meccanismi che cooperano

Meccanismo	Ruolo
Gruppo programmatori <code>chmod 2770</code>	Definisce chi può accedere alla directory SGID + <code>rw-rwx---</code> (solo group e owner, nessun altro)
SGID sulla directory	I file creati dentro ereditano il gruppo programmatori
<code>umask 0002</code> nel <code>.bashrc</code>	I nuovi file hanno permessi <code>664</code> — il gruppo può scrivere

Questi quattro elementi si completano a vicenda. Togliere uno qualsiasi rompe la collaborazione: - Senza SGID: i file hanno il gruppo primario del creatore, l'altro non può scriverci - Senza `umask 0002`: i file hanno permessi `644`, il gruppo può solo leggere - Senza `chmod 770`: la directory è accessibile a chiunque (o a nessuno) - Senza il gruppo condiviso: non esiste un soggetto comune a cui dare i permessi

Connessione con Security

- Una directory 777 al posto di 2770 rimuove la protezione verso gli esterni — qualsiasi utente del sistema può accedere, modificare, cancellare
 - Il SGID su eseguibili (non directory) è un vettore di privilege escalation: se un programma con SGID è vulnerabile, un attaccante può acquisire i privilegi del gruppo proprietario
 - `find / -perm -2000 -type f` elenca tutti i file con SGID attivo sul sistema — uno dei controlli base di un audit
-

Riepilogo comandi

Comando	Funzione
<code>ls -la /etc/skel</code>	Vedi il template delle home utente
<code>cat /etc/default/useradd</code>	Default di useradd
<code>grep -v '^#' /etc/login.defs \ grep -v '^\$'</code>	Policy globali account (senza commenti)
<code>echo "umask 0002" >> ~/.bashrc</code>	Imposta umask persistente per un utente
<code>sudo groupadd nome</code>	Crea un nuovo gruppo
<code>sudo usermod -aG gruppo utente</code>	Aggiunge utente al gruppo
<code>sudo chgrp gruppo directory</code>	Assegna gruppo proprietario alla directory
<code>sudo chmod 02770 directory</code>	SGID + rwxrwx—
<code>chmod g+s directory</code>	Imposta SGID in simbolico
<code>newgrp gruppo</code>	Attiva un nuovo gruppo nella sessione corrente senza re-login
<code>find / -perm -2000 -type f</code>	Lista file con SGID attivo (audit)

Collegati

- [\[\[Appunti_modulo2B\]\]](#)
- [\[\[appunti_modulo2B_lab_utenti_permessi_file\]\]](#)

Hub: [\[\[master_map_studio\]\]](#) · [\[\[glossario_sysadm\]\]](#) · [\[\[concept_maps\]\]](#) · [\[\[troubleshooting_vm\]\]](#)
· [\[\[metodo_studio_esami_pratici\]\]](#)